Documentation for `simple_ace_mg_full.py`

Multigroup ACE Generation, ACE → HDF5 Conversion, and MCNP Geometry → Gmsh Meshing

Luke Wood

December 8, 2025

Contents

1 Introduction **2**

2 High-Level Capabilities **2**

3 Dependencies and Environment **3**
3.1 Python and Core Libraries . . . 3
3.2 Optional Libraries . . . 3

4 JSON → ACE Workflow **3**
4.1 Running the Script . . . 3
4.2 JSON Schema Overview . . . 4
4.3 Example JSON Snippet . . . 5
4.4 ACE and XSDIR Output . . . 6

5 ACE → HDF5 (OpenMC mgxs) Workflow **6**
5.1 Single ACE File . . . 6
5.2 Multiple ACE Files per Problem . . . 7

6 Verification Tree Scan + Optional Gmsh Meshes **7**
6.1 ACE Scanning and HDF5 Libraries . . . 7
6.2 Optional Mesh Generation with Gmsh . . . 7

7 Internal Data Model and Implementation Details **8**
7.1 MGData . . . 8
7.2 ACE Construction (NXS, JXS, XSS) . . . 9
7.3 ACE Reading . . . 9
7.4 Summary and Consistency Checks . . . 9

8 Typical End-to-End Workflow **10**

9 Conclusion **10**

1 Introduction

This document explains how to use the Python script `simple_ace_mg_full.py`, which is a fairly complete Python port of LANL's `simple_ace_mg.pl` utility for generating simple multigroup ACE files and extends it with:

- JSON → multigroup ACE generation (MCNP format).
- ACE → OpenMC-style multigroup HDF5 (`mgxs.h5`).
- Automatic scanning of a verification directory tree (e.g. MG01–MG75) and per-problem HDF5 library creation.
- Optional geometry meshing for each problem using Gmsh: either from existing `.geo`/`.step` files or via a minimal MCNP-surface parser (PX/PY/PZ, spheres, cylinders).

The script is intended for verification and educational use, where one wants to quickly generate simple multigroup cross section sets and feed them to MCNP, OpenMC, or OpenSn.

2 High-Level Capabilities

The script supports three main workflows from the command line:

1. **JSON configuration → multigroup ACE**
   - Input: a user-defined JSON file describing a multigroup cross-section set.
   - Output:
     - A multigroup ACE file (e.g. `mg01xsecs`).
     - A mini `xsdir` file (`xsdir_mcnp6.3`) for MCNP.
     - A human-readable summary file (`mgXXxsecs.summary.txt`).
     - A small MCNP input snippet for using the ACE table.
2. **ACE → OpenMC-style HDF5 mgxs library**
   - Input: one or more multigroup ACE files.
   - Output: `mgxs.h5` libraries compatible with OpenMC's multigroup cross section format (isotropic scattering).
3. **Verification tree scan + optional Gmsh meshes**
   - Input: a root directory containing many problem subdirectories and ACE tables (e.g. LANL keff verification benchmarks).
   - Output:
     - One `mgxs.h5` per problem directory, containing all isotopes found there.
     - Optionally, one `geom.msh` Gmsh mesh per problem (from `.geo`/`.step` or MCNP input).

3 Dependencies and Environment

3.1 Python and Core Libraries

You need:

- Python 3.8+ (the script uses type hints and `dataclasses`).
- Standard library modules: `argparse`, `os`, `json`, `subprocess`, `datetime`, etc.

3.2 Optional Libraries

Depending on the workflow:

- **HDF5 writing** (ACE → mgxs):
  - `h5py` (`pip install h5py`).
  - `numpy` (for the combined mgxs writer).
- **Gmsh meshing**:
  - Gmsh Python module: `pip install gmsh`.
  - Optional external Gmsh executable (e.g. `gmsh.exe` on Windows) if you want to mesh from `.geo` or `.step` files via the CLI.

The JSON → ACE workflow works with only the Python standard library (no extra packages needed).

4 JSON → ACE Workflow

4.1 Running the Script

The simplest usage to go from JSON to ACE is:

```
python simple_ace_mg_full.py --config ex3.json
```

This will:

1. Parse the JSON file.
2. Build the internal multigroup structure (energies, cross sections, scattering matrix, etc.).
3. Create:
   - A multigroup ACE file (default name like `mg01xsecs`, `mg02xsecs`, etc., based on NG).
   - A compact `xsdir_mcnp6.3` file describing that table.
   - A `.summary.txt` file with a compact table of groupwise total, capture, scatter, fission, ν, and χ.
   - A short MCNP snippet showing how to use the table.

4.2 JSON Schema Overview

The script expects a JSON file with the following main fields (names are flexible; several aliases are supported for each quantity):

- `"zaid"` (string, required)
  e.g. `"99902.01m"`. Must end in `m`.
- `"awr"` (float, optional)
  Atomic weight ratio. Default is `1.0`.
- `"tmp"` (float, optional)
  Temperature in either kT (MeV) or Kelvin. If `tmp > 1`, it is interpreted as Kelvin and converted internally to MeV. Default: `2.5301e-8` MeV (room temperature).
- `"comment"` (string, optional)
  Free text describing the ACE table; truncated to 70 characters for the ACE header.
- `"e_bounds"` (array, required)
  Energy group boundaries (MeV), length NG+1, in *either* ascending or descending order. Internally the script enforces strictly descending order. Aliases: `"erg"`, `"energies"`, `"energy_grid"`.
- `"groups"` or `"NG"` (int, optional)
  Number of energy groups. If omitted, defaults to `len(e_bounds) - 1`.
- `"micro_xs"` (bool, optional)
  If `true`, you are providing *microscopic* cross sections. The script multiplies them by the `"atom_density"` to obtain macroscopic values.
- `"atom_density"` (float, optional)
  Atom density (e.g. in atoms/barn-cm). Required if `"micro_xs" = true`.

Cross-Section Arrays

All cross-section arrays are per-group (length NG) unless otherwise noted, and they may be:

- omitted (treated as all zeros),
- length 1 (constant value replicated to all groups),
- or length NG (groupwise specified).

Recognized keys:

- **Total** Σ_t:
  - Keys: `"sigma_t"`, `"Sigma_t"`, `"sigt"`.
- **Capture** (non-fission absorption) Σ_c:
  - Keys: `"sigma_c"`, `"Sigma_c"`, `"Sigma_a"`, `"sigc"`.
- **Fission** Σ_f:

  - Keys: `"sigma_f"`, `"Sigma_f"`, `"sigf"`.

- **Average multiplicity ν**:
  - Key: `"nu"`.

- **Fission spectrum χ**:
  - Key: `"chi"`.
  - If any fission cross section is non-zero, the script normalizes χ so that $\sum_g \chi_g = 1$.
  - If no fission is present, χ and ν are forced to zero.

- **Scattering matrix $\Sigma_{s,g\to g'}$**:
  - Keys: `"sigma_s"`, `"Sigma_s"`, `"sigs"`.
  - Accepted forms:
    * A flat array of length N_G^2 (row-major: initial group index varies slowest).
    * A nested list `[[...], [...], ...]` of shape NG×NG.

- **P1 scattering (anisotropy) $\Sigma_{s,1}$**:
  - Keys: `"sigma_s_p1"`, `"Sigma_s_p1"`, `"sig1"`.
  - Same shape options as Σ_s.
  - If all P1 entries are zero, scattering is treated as isotropic.

If Σ_t is not provided or is all zeros, the script automatically fills it as

$$\Sigma_t = \Sigma_c + \Sigma_f + \sum_{g'} \Sigma_{s,g\to g'}.$$

4.3 Example JSON Snippet

A simple two-group example (with macroscopic cross sections):

```
{
  "zaid": "99953.01m",
  "awr": 236.947,
  "tmp": 2.5301e-08,
  "comment": "Debug 2-group case, no fission",
  "e_bounds": [20.0, 0.625e-6, 1.0e-5],

  "sigma_c": [0.2, 0.1],
  "sigma_f": [0.0, 0.0],
  "sigma_s": [
    [0.8, 0.0],
    [0.7, 0.0]
  ],

  "nu": [0.0, 0.0],
  "chi": [0.0, 0.0],

  "micro_xs": false
}
```

4.4 ACE and XSDIR Output

After running `--config`, you obtain:

- An ACE file containing:
  - Two header lines (ZAID, AWR, kT in MeV, date; and comment).
  - Four lines of zeros (matching the original Perl format).
  - NXS array (16 integers).
  - JXS array (32 integers).
  - XSS array (double precision, contains energy midpoints, widths, xsections, scattering matrix, and angular data).
- A minimal `xsdir_mcnp6.3` file of the form:

```
atomic weight ratios
99953 236.947000
directory
 99953.01m 236.947 mg02xsecs 0 1 1 24 0 0 2.53010e-08
```

- A printed MCNP snippet:

```
mgopt f 2
xs1 99953.01m 20 mg02xsecs 0 1 1 24 0 0 2.5301e-08
m1 99953.01m 1.0
```

5 ACE → HDF5 (OpenMC mgxs) Workflow

5.1 Single ACE File

To convert a single ACE file into OpenMC-style `mgxs.h5`:

```
python simple_ace_mg_full.py --ace mg01xsecs
```

If `--h5` is omitted, the output is `mg01xsecs.h5`. You can also specify an explicit name:

```
python simple_ace_mg_full.py --ace mg01xsecs --h5 mg01xs.h5
```

Internally the script:

1. Reads the ACE header, NXS, JXS, and XSS arrays.
2. Reconstructs the energy grid from the ACE group midpoints and widths.
3. Extracts total, fission, ν, χ, capture, and the P0 scattering matrix.
4. Creates an HDF5 file conforming to OpenMC's multigroup library layout (one nuclide group for the ACE table).

The HDF5 structure includes:

- `filetype = "mgxs"`.

- `group structure` dataset (energies in eV, ascending).
- One group named by ZAID (or `--library-name` if provided).
- Temperature group `"<T>K"` computed from `tmp` (kT MeV).
- Datasets: `total`, `absorption`, optional `fission`, `chi`, `nu-fission`, and P0 scattering in a simple matrix form.

5.2 Multiple ACE Files per Problem

For verification problems with multiple isotopes, the script provides a "combined library" writer:

- All ACE tables in a given problem directory are read into `MGData` objects.
- The script checks:
  - Same number of groups NG.
  - Identical energy grids.
  - Same temperature (kT) for all isotopes.
- A single `mgxs.h5` is created with one HDF5 group per ZAID.

6 Verification Tree Scan + Optional Gmsh Meshes

6.1 ACE Scanning and HDF5 Libraries

To scan a verification tree and convert all problems at once:

```
python simple_ace_mg_full.py \
  --verification-root "C:\path\to\problems" \
  --outdir "C:\path\to\problems_h5"
```

The script:

1. Walks `--verification-root` recursively.
2. Tries to interpret each file as a multigroup ACE table.
3. Groups all ACE tables in a given problem directory.
4. Writes one `mgxs.h5` per problem under `--outdir`, with the same subdirectory structure.

6.2 Optional Mesh Generation with Gmsh

If `--make-mesh` is added:

```
python simple_ace_mg_full.py \
  --verification-root "C:\path\to\problems" \
  --outdir "C:\path\to\problems_h5" \
  --make-mesh --gmsh "gmsh"
```

then for each problem directory:

1. If a `.geo` or `.step`/`.stp` file exists, the script calls the Gmsh executable. For example: `gmsh_exe geom.geo -3 -o geom.msh`.
2. Otherwise, if an MCNP input file is found (`.i`, `.inp`, or `.mcnp`), the script parses its surface cards and uses the Gmsh Python API to construct a basic 3D region and mesh it.
3. A file `geom.msh` is written next to `mgxs.h5` in the output tree.

Geometry Detection. The script infers a high-level geometry type from the problem name suffix:

- `-IN`: Infinite medium (uses a bounding box).
- `-SL` or `-ISLC`: Slab-type problems.
- `-CY`: Cylindrical problems.
- `-SP`: Spherical problems.

This guides whether it builds a box, concentric spheres, or concentric cylinders.

Supported MCNP Surfaces (for Python-based meshing). The surface parser supports:

- Planes: `PX`, `PY`, `PZ`.
- Spheres: `SO R`, or general spheres `S/SPH x0 y0 z0 R`.
- Infinite cylinders: `CZ`, `CX`, `CY`.
- Finite cylinders: `RCC x0 y0 z0 dx dy dz R`.

It ignores comments and tokens after a `$`-sign.

7 Internal Data Model and Implementation Details

7.1 MGData

The core data container is a `@dataclass` called `MGData`:

- `zaid`, `za`, `awr`, `tmp`, `comment`
- `NG`, `erg` (NG+1), and groupwise arrays: `sigt`, `sigc`, `sigf`, `nu`, `chi`
- `sigs`: scattering matrix as a flat list of length N_G^2.
- `sig1`: P1 scattering (same shape) for anisotropy (optional).
- `nbins`: number of equiprobable angular bins if anisotropic.

7.2 ACE Construction (NXS, JXS, XSS)

The function `build_ace(mg)`:

1. Decides whether scattering is isotropic (all `sig1` ≈ 0) or anisotropic.
2. Fills the ACE `NXS` and `JXS` arrays according to the LANL `simple_ace_mg.pl` convention.
3. Builds the `XSS` array containing:
   - Group midpoints and widths (descending energies).
   - Total, fission, ν, χ, capture.
   - P0 scattering matrix (flattened).
   - Optional angular information for P1 (equiprobable μ-bins).
4. Returns `nxs, jxs, xss`.

7.3 ACE Reading

The function `read_mg_ace(ace_file)`:

1. Reads the two header lines to obtain `zaid`, `awr`, `tmp` (kT in MeV), and `date`.
2. Skips four lines of zeros.
3. Reads 16 integers into `NXS` and 32 into `JXS`.
4. Reads `N = NXS[1]` floating-point values into `XSS`.
5. Reconstructs:
   - Energy grid from midpoints and widths.
   - Total, fission, ν, χ, capture.
   - P0 scattering matrix.
6. Returns a new `MGData` along with `nxs, jxs, xss`.

7.4 Summary and Consistency Checks

The `write_summary` function:

- Writes a table of groupwise: $(E_{\text{hi}}, E_{\text{low}}, \Sigma_t, \Sigma_c, \Sigma_s, \Sigma_f, \nu, \chi)$.
- Recalculates $\Sigma_s = \sum_{g'} \Sigma_{s,g\to g'}$ from the scattering matrix.
- Checks that $\Sigma_t \approx \Sigma_c + \Sigma_f + \Sigma_s$ within a small tolerance.

8 Typical End-to-End Workflow

A typical usage pattern for verifying a simple problem:

1. Create a JSON file specifying the energy grid and cross sections (possibly from hand calculations or a code).
2. Run:

```
python simple_ace_mg_full.py --config ex3.json
```

3. Plug the generated ACE table, `xsdir_mcnp6.3` entry, and MCNP `mgopt/xs1/m1` snippet into an MCNP input file and run MCNP to obtain fluxes or keff.
4. Once satisfied, convert the ACE table to an OpenMC-style multigroup library:

```
python simple_ace_mg_full.py --ace mg01xsecs --h5 mg01xs.h5
```

5. Use the resulting `mgxs.h5` in OpenMC or OpenSn for deterministic/Monte Carlo comparisons.

9 Conclusion

The script `simple_ace_mg_full.py` provides a complete workflow for *simple* multigroup verification problems:

- From JSON definitions to MCNP-usable ACE tables.
- From ACE tables to OpenMC-style multigroup HDF5.
- From verification trees (with many problems) to structured HDF5 libraries and optional Gmsh meshes.

Because it closely follows LANL's original `simple_ace_mg.pl` logic, it is both transparent and convenient for debugging small multigroup test cases and serves as a bridge between MCNP and modern Python-based neutronics workflows.